

01 경력 요약

C/C++ 기반 소프트웨어 엔지니어로 17년 이상 임베디드 시스템에서 시작하여 의료영상, 산업제어를 거쳐 Unreal Engine 3D 제작도와 AI/LLM 응용까지 도메인을 확장하며 제품을 개발해왔다. 분석, 설계, 구현, 검증, 운영 등 제품 전 과정을 고려하여 작업의 디테일을 정하고 의견을 교환하는 업무 스타일을 지향한다. 최근 3D 콘텐츠 제작 프로젝트에서 Camera Direction System 설계와 LLM/AI 모델 연동 기능을 개발했다.

기간	회사	역할	주요 프로젝트
2022.11 — 2026.03	시나몬스튜디오	시니어 프로그래머	CinevStudio — Unreal Engine 3D 영상 제작도구
2020.04 — 2022.11	인그레디언트	시니어 프로그래머	MediLabel On-premise — 의료영상 Annotation 도구
2017.01 — 2020.03	하이버스	개발팀장 과장	GIPAM3000 전면 HMI — 전력보호감시장치
2014.06 — 2016.12	하이버스	프리랜서 개발 (MDEC SW)	CMCIS MDEC — 의료장비 프로토콜 게이트웨이
2012.06 — 2014.03	윌메드	개발팀 대리	WillCeph, Dentio — 치과 진단장비 SW
2010.04 — 2012.06	하이버스	개발팀 사원	PDK3200, UFSN — 산업장비/센서네트워크
2008.06 — 2010.03	바텍	개발팀 사원	PaX-Primo — 치과용 X선 장비 SW

02 기술 역량

언어

C/C++을 주 언어로 사용. 도메인에 따라 다양한 프레임워크와 플랫폼 위에서 C++ 활용 — Qt (크로스플랫폼 UI), Unreal Engine (Camera System, Sequencer), VTK (의료영상 시각화), MFC (Windows GUI), Embedded C++ (임베디드 시스템).

플랫폼

Unreal Engine 4/5: Camera System, Sequence 편집, AI 연동, UE4→UE5 마이그레이션
Qt 5/6: 의료영상 UI, 산업 HMI, VTK 통합, 교차 플랫폼
MFC: 의료영상 Viewer, Ribbon UI, GDI+
Embedded Linux: 크로스컴파일, ARM 타겟, 디바이스 연동

도메인

의료영상 (DICOM, CT/MRI/Panorama, 2D/3D Visualization, Image Chain)
Unreal Engine (UE4/5, Sequencer, Camera System, MovieRenderQueue)
임베디드 (크로스컴파일, UART/USB/TCP/IP, 프로토콜 변환)
AI/LLM (Function Calling, TensorFlow 통합, AI→제작데이터 변환)

개발 환경

Git · GitLab · Perforce · SVN · CMake · Unreal Build System · CI/CD · Source Insight 정적 분석

CinevStudio

2022.11 — 2026.03 (3년 5개월)

시나몬스튜디오 · 시니어 프로그래머

C++ Unreal Engine 4/5 LLM Function Calling Git

CINEV 서비스의 3D 영상 콘텐츠 제작 엔진. 사용자 프롬프트 → LLM 장면 정의 → Unreal Engine 3D Scene → 렌더링 파이프라인의 핵심 담당.

- **Camera Direction System:** 영화 촬영 이론 기반 Shot 정의, AutoCamera 규칙 시스템, Sequence Keyframe 구조로 일반화
- **LLM/AI 연동:** Function Calling으로 AI 출력을 구조화된 제작 명령어로 변환, 편집 가능한 Sequence 데이터 통합
- **UE5 마이그레이션:** SequencePlayer 동작 방식 변경 대응, ForceEvaluate 우회 구현
- **CLI 렌더링 파이프라인:** MovieRenderQueue CLI 구동, Web API 연동, Motion Blur/Lumen Snapshot 문제 해결
- **Grouped Action System:** GameplayTags 기반 Plugin Asset Type, POC 완료 후 팀 이관

MediLabel On-premise

2020.04 — 2022.11 (2년 8개월)

인그래디언트 · 시니어 프로그래머 (개발 중단으로 종료)

C++ Qt VTK TensorFlow REST API

딥러닝 기반 의료영상(CT/MRI) 라벨링 소프트웨어. 2D Slice Annotation + 3D Volume 편집, TensorFlow AI 모델 통합.

- **2D-3D 데이터 일관성:** 2D Polygon ↔ 3D Volume 간 양방향 변환 구조, 의사/라벨러에 익숙한 2D 조작 방식 유지
- **AI 변환 레이어:** TensorFlow 세그멘테이션 마스크 → Label 데이터 변환 모듈 분리, 모델 교체에 독립적
- **3D 편집:** VTK 기반 Viewer, Isocontour(CT value 자동 추출), Sculpting(3D 브러시 편집)
- **Cloud 연동:** REST API 기반 온프레미스-Cloud 하이브리드 구조

GIPAM3000 전면 HMI

2017.01 — 2020.03 (3년 3개월)

하이버스 (LS Electric 외주) · 개발팀장 과장

C++ Qt Embedded Linux MODBUS-TCP SVN

LS Electric GIPAM3000 디지털 전력보호감시장치의 전면 HMI Application. 34종 보호 요소 설정/계측/이력 조회.

- **스크립트 기반 동적 UI:** CSV 리소스 파싱 → 메뉴 계층 동적 구성 → MODBUS 자동 연결. 보호 요소 추가 시 CSV만 갱신
- **하드웨어 없는 MODBUS 개발:** 시뮬레이션 서버 → 기존 장비 → 실제 GIPAM3000 순 단계적 검증
- **임베디드 최적화:** 프레임버퍼 Tearing/잔상 문제 해결, Watchdog 기반 장애 복구, 메모리 누수 최소화
- **신뢰성 검증:** Source Insight 정적 분석 정기 수행, 고객사 QA 협업 온도/습도/Surge 환경 테스트

04 기타 프로젝트

CMCIS MDEC

2014.06 — 2016.12 · 프리랜서

국책과제 의료장비 프로토콜 게이트웨이. 6종 장비 데이터 수신 → CMCIS 표준 변환. Linux, UART/USB, TCP/IP.

WillCeph

2013.06 — 2014.03 · 월메드

2D Cephalometric 분석 SW. MFC Ribbon, GDI+. Landmark Annotation, VTO/STO 시뮬레이션, Growth Forecast.

Dentio

2012.06 — 2013.06 · 월메드

파노라마+세팔로 통합 X선 장비. Teledyne Dalsa CMOS 협업, Image Chain, Calibration Application.

PDK3200

2011.08 — 2012.06 · 하이버스

OLED/LCD Tester & Demo Kit. 임베디드 + Windows 병행 개발. Pattern Generator (픽셀 단위 dot 출력).

UFSN

2010.04 — 2011.08 · KETI 외주

지하시설물 센서 네트워크. PLC 통신, 유무선 이중화, lighttpd+PHP 웹서비스. 논문 게재.

PaX-Primo

2009.10 — 2010.03 · 바텍

치과용 파노라마 X선 장비. WinCE 5.0, FPGA Frame Grabber, Ethernet 단일 인터페이스.

05 엔지니어링 접근 방식

01 문제를 해결할 레이어의 결정

동일한 기능 요구라도 데이터 모델, 통신 인터페이스, 사용자 경험 중 어느 계층에서 해결하느냐에 따라 시스템의 유지보수성과 확장성이 달라진다. 구조적 결정이 시간이 지나면서 시스템의 변경 비용을 결정한다.

CinevStudio: Camera 데이터를 Sequence Keyframe 구조로 일반화하여 UE 업데이트 영향에서 데이터 모델 분리

02 변경 비용을 고려한 구조 설계

"이 기능이 시간이 지나면서 어떻게 변할 것인가?" — 엔진 업데이트, 모델 교체, 요구사항 변경 등 변화의 방향을 예측하고 영향 범위를 최소화하는 구조를 선택한다.

MediLabel: AI 모델 출력 변환 레이어를 분리하여 TensorFlow→PyTorch 교체에도 코어 영향 없음

03 AI 결과를 사람이 검증할 수 있는 형태로

AI 생성 결과를 그대로 사용하지 않고, 편집 가능한 제작 데이터로 변환하여 사람의 판단을 중간에 넣을 수 있도록 한다. AI 출력의 불완전성을 구조적으로 보완한다.

CinevStudio: LLM 출력 → Sequence 데이터 변환 → 편집 가능. MediLabel: AI 마스크 → Polygon Label → 사용자 수정

AI 경험을 LLM 기반 제작 파이프라인과 학습모델 통합의 두 축으로 나눈다.

LLM 기반 제작 파이프라인 (CinevStudio)

CinevStudio에서는 LLM을 제품 경험의 전면에 두었다. 사용자 텍스트 프롬프트 → LLM 장면 정의 → Unreal Engine 3D Scene → 렌더링으로 이어지는 파이프라인에서, LLM 기능의 REST API 설계와 Application 측 처리 구조를 담당했다.

REST API 설계 및 Data Schema 협의. LLM 기능을 Application에서 사용하기 위해 REST API 형태로 인터페이스를 정의했다. 요청/응답 데이터 구조(Schema)를 설계하고, AI 엔지니어 및 기획팀과 협의하여 API 명세를 확정했다. LLM 출력(JSON)은 Sequence 생성 명령어로 매핑되어 3D Scene 구성의 입력으로 사용되었다.

AI 결과의 중간 표현. LLM 응답을 그대로 사용하지 않고, 편집 가능한 Sequence Keyframe 데이터로 변환하는 중간 레이어를 두었다. 이를 통해 AI 생성 결과를 사용자가 검토하고 수정할 수 있도록 했다. AI 출력의 불완전성을 구조적으로 보완하는 접근으로, 제작자의 의도가 반영된 최종 Scene을 구성할 수 있게 했다.

동기 HTTP 통신 처리. Unreal Engine의 HTTP 모듈은 비동기 콜백 기반이지만, LLM 호출 후 결과를 기다려 다음 처리로 이어져야 하는 파이프라인 특성상 동기(blocking) 방식이 필요했다. 비동기 요청의 콜백에서 결과 도착까지 스레드를 대기시키는 래퍼를 구현하여 순차적 처리 흐름을 유지했다.

학습모델 통합 (MediLabel)

MediLabel에서는 TensorFlow 기반 의료영상 분할 모델의 출력을 제품 내 Label 데이터로 통합하는 변환 레이어를 설계했다. LLM과 달리 이 프로젝트는 학습모델의 출력(세그멘테이션 마스크)을 Polygon/Volume Label로 변환하여 Annotation 도구의 기능으로 제공하는 데 중점을 두었다.

AI 모델 독립적 변환 레이어. TensorFlow 세그멘테이션 마스크를 Label 데이터로 변환하는 모듈을 별도 레이어로 분리했다. 노이즈 제거, 경계 스무딩, Label 우선순위 처리를 포함하여, 추후 다른 AI 모델(PyTorch, ONNX 등)로 교체되어도 제품 코어에 영향을 주지 않도록 했다.

2D-3D 데이터 일관성. AI 모델 출력은 3D Volume(픽셀 단위 마스크)이지만, 실제 라벨러가 조작하는 단위는 2D Slice Polygon이었다. 2D Polygon과 3D Volume Label 간 양방향 변환 구조를 정의하여, AI 생성 결과를 익숙한 2D 조작 방식으로 편집할 수 있도록 했다.

전 직장에서도 Agent 기반 개발 방식이 사내 트렌드로 자리잡기 시작하면서 관련 방법론을 습득하고 현업에 적용하기 시작했다. 퇴사 후 개인 Unreal Engine 프로젝트에서 Agent 기반 개발 환경을 직접 구성하여 운영 중이다.

문서 기반 Agent 행동 정의

Agent의 행동을 계층화된 문서로 정의했다. **AGENTS.md**에 핵심 원칙(Simplicity First, Surgical Changes, Goal-Driven Execution)을 두고, 구체적인 업무 절차는 **Doc/Process/** 디렉토리 아래 개별 문서로 분리했다. 상세 워크플로우가 AGENTS.md에 비대해지면 Doc/Process/로 이동시키는 규칙을 통해 문서 간 책임 범위를 유지했다.

- **AGENTS.md**: 핵심 원칙, 프로젝트 컨텍스트, 코딩 컨벤션
- **Doc/Process/**: 문서 관리, Git 운영, Safeguard, 세션 복원 절차
- **Doc/Planning/**: 기획, 백로그, 향후 작업
- **Doc/Daily/**: 작업 일지 (Agent Work / User Work 분리)

Goal-Driven Execution

모든 작업을 검증 가능한 목표로 변환하는 방식을 적용했다. 단순한 지시("버그 수정") 대신 재현 조건과 확인 기준을 명시("재현 테스트 작성 → 통과 → 수정 → 테스트 재통과")하여 Agent가 독립적으로 반복 작업을 수행할 수 있도록 했다. 다단계 작업은 단계별 확인 조건을 사전에 정의하고 순차적으로 실행했다.

작업 일지와 세션 복원

매 작업 세션의 결과를 **Doc/Daily/{날짜}/notes.md**에 기록한다. Agent가 직접 수행한 코드 편집/분석/문서 갱신은 **Agent Work**, Agent 안내에 따라 사용자가 직접 실행한 Unreal Editor 조작(BP 편집, PIE 튜닝)은 **User Work**로 분리하여 기록한다. 각 작업 단위는 **목표 → 수정 파일 목록 → 검증 결과** 순서로 정리한다.

새 세션 시작 시 SESSION_START.md에 정의된 복원 순서(AGENTS.md → Doc/README.md → 최신 Daily notes → Planning 문서)를 통해 일관된 컨텍스트를 복원한다.

Safeguard 자동화

반복되는 실패 패턴을 문서화 대신 자동화된 검사 도구로 방어한다. Git pre-commit hook, Python 검증 스크립트를 통해 오류 발생 → 원인 분석 → Safeguard 생성 → 시스템 통합 → 모니터링 사이클을 운영한다. 단순한 규칙 나열이 아니라 실행 가능한 검사로 실패를 차단하는 접근이다.

지식 그래프를 통한 코드베이스 분석

코드베이스 분석에 knowledge graph(graphify)를 활용한다. 소스 코드를 AST 기반 지식 그래프로 구축하고, 파일 검색(grep)보다 그래프 탐색을 우선하여 모듈 간 관계와 의존성을 파악한다. 코드 수정 후 그래프를 갱신하여 최신 상태를 유지한다.